

Reviewer Pack — Minimal Order of Hodge Decomposition over Communication Comp...

Entry 877cb94960d4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 14:47:22 UTC

Minimal Order of Hodge Decomposition over Communication Complexity for XOR-AND Trees

Entry ID: 877cb94960d4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 14:47:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Decomposition) **Field B** (complexity object): Communication Complexity (XOR-AND Trees)

Statement:

[‘For any given n -bit input, the Hodge decomposition of the corresponding XOR-AND tree has a minimal order that is at least $\Omega(n^{\{1/3\}})$ and at most $O(n^{\{4/5\}})$.’, ‘A single counterexample with an Hodge decomposition of minimal order less than $\Omega(n^{\{1/3\}})$ or greater than $O(n^{\{4/5\}})$ would falsify the conjecture.’, ‘For a given XOR-AND tree, compute its Hodge decomposition and analyze the orders of its components to determine if they fall within the proposed bounds.’]

Rationale (proposer’s reasoning):

[‘The Hodge decomposition provides a structured way to understand the complexity of geometric objects. By applying this to XOR-AND trees, we can potentially uncover new insights into communication complexity.’, ‘Hodge structures have been used in algebraic geometry and complex analysis, but their application to XOR-AND trees is novel and could reveal deep connections between these fields.’, ‘The proposed bounds are based on empirical observations of the typical structure of XOR-AND trees and should be testable with current computational methods.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 723588b114a06aa7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 25 of the 30 random seeds, the distribution of minimal Hodge decomposition orders lies within the bounds $\Omega(n^{\{1/3\}})$ and $O(n^{\{4/5\}})$. The conjecture is falsified if any seed shows a minimal order outside these bounds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Decomposition" AND "Communication Complexity" AND "XOR-AND Trees" - "Algebraic Geometry" AND "minimal order Hodge decomposition" AND "communication complexity" - "XOR-AND trees" AND Hodge AND "order analysis"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor_and_tree(n):
        if n == 1:
            return [0, 1]
        else:
            left = xor_and_tree(n // 2)
            right = xor_and_tree(n - n // 2)
            return [left[i] ^ right[i] for i in range(len(left))]

    def hodge_decomposition(tree):
        if len(tree) == 1:
            return [1]
        else:
            left_hodge = hodge_decomposition(tree[:len(tree)//2])
            right_hodge = hodge_decomposition(tree[len(tree)//2:])
            return [left_hodge[i] + right_hodge[i] for i in range(len(left_hodge))]

    n = random.randint(5, 40)
    tree = xor_and_tree(n)
    hodge_order = len(hodge_decomposition(tree))

    metric_name = "hodge_order"
    metric_value = hodge_order
    instances_tested = 1
    conjecture_holds = (n**(1/3) <= hodge_order <= n**(4/5))
```

```

counterexample = "" if conjecture_holds else f"hodge_order={hodge_order}, expected=[{int(n**(1/3))}]"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = [run_trial(seed) for seed in seeds]

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_dev:.4f} support_fraction={support_fraction:.4f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{r['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4995bacc.py", line 68, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4995bacc.py", line 68, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from verifying whether the conjecture is supported or falsified. | next: Re-run the test to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11836
2	propose	ollama_remote	glm4:latest	0	0	11137
3	propose	ollama_remote	glm4:latest	0	0	10320
4	propose	ollama_remote	glm4:latest	0	0	11134
5	preregistration	ollama_remote	glm4:latest	0	0	6088
6	novelty	ollama_remote	glm4:latest	0	0	4726
7	novelty	ollama_remote	glm4:latest	0	0	5374
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12371
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8931
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9288
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8891
12	judge	ollama_remote	glm4:latest	0	0	8153

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108248 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/877cb94960d4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/877cb94960d4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/877cb94960d4.tar.gz` (if generated)